LUMUT

INSTITUT KEMAHIRAN MARA LUMUT

DIPLOMA TEKNOLOGI KOMPUTER (ANALISA DATA BESAR)

(DFD21043)

COMPUTER MAINTENANCE AND SUPPORT

LAB : ARDUINO IDE

PREPARED BY:

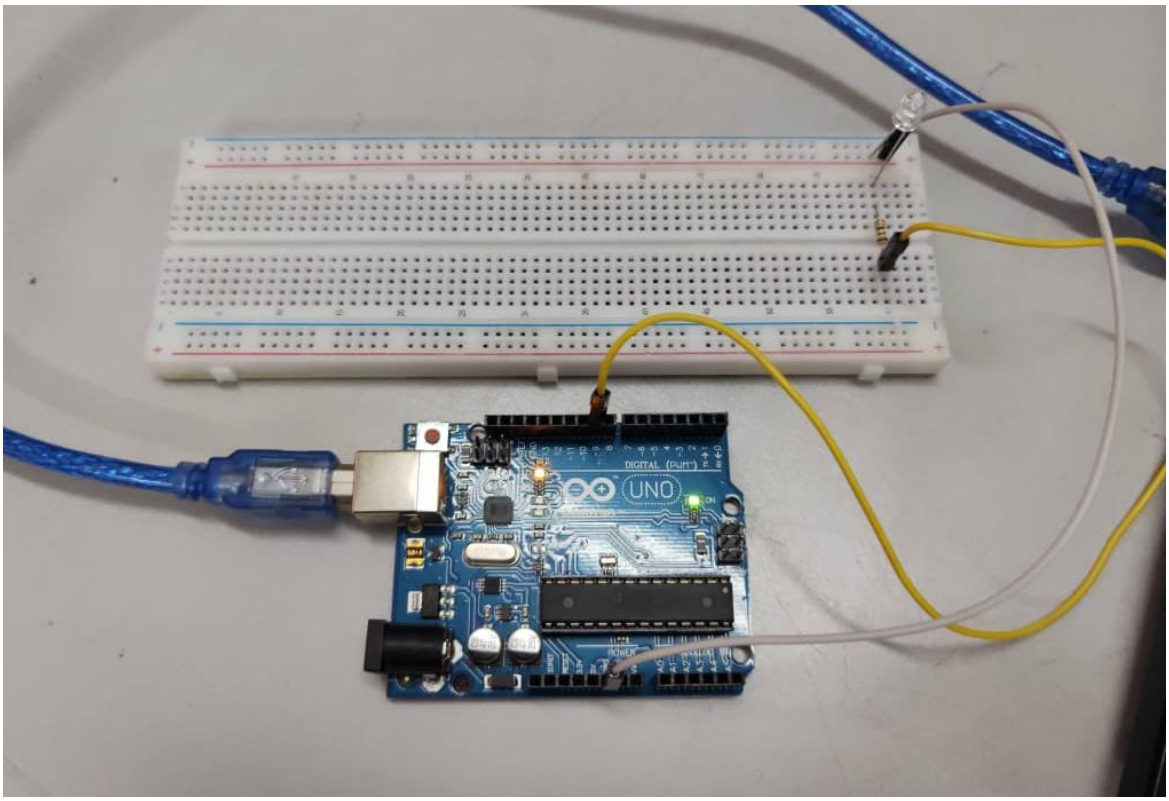
MOHAMAD HAIRI BIN ABDUL AHMAD

PREPARED FOR:

NAJUWA BINTI ABD LATIP

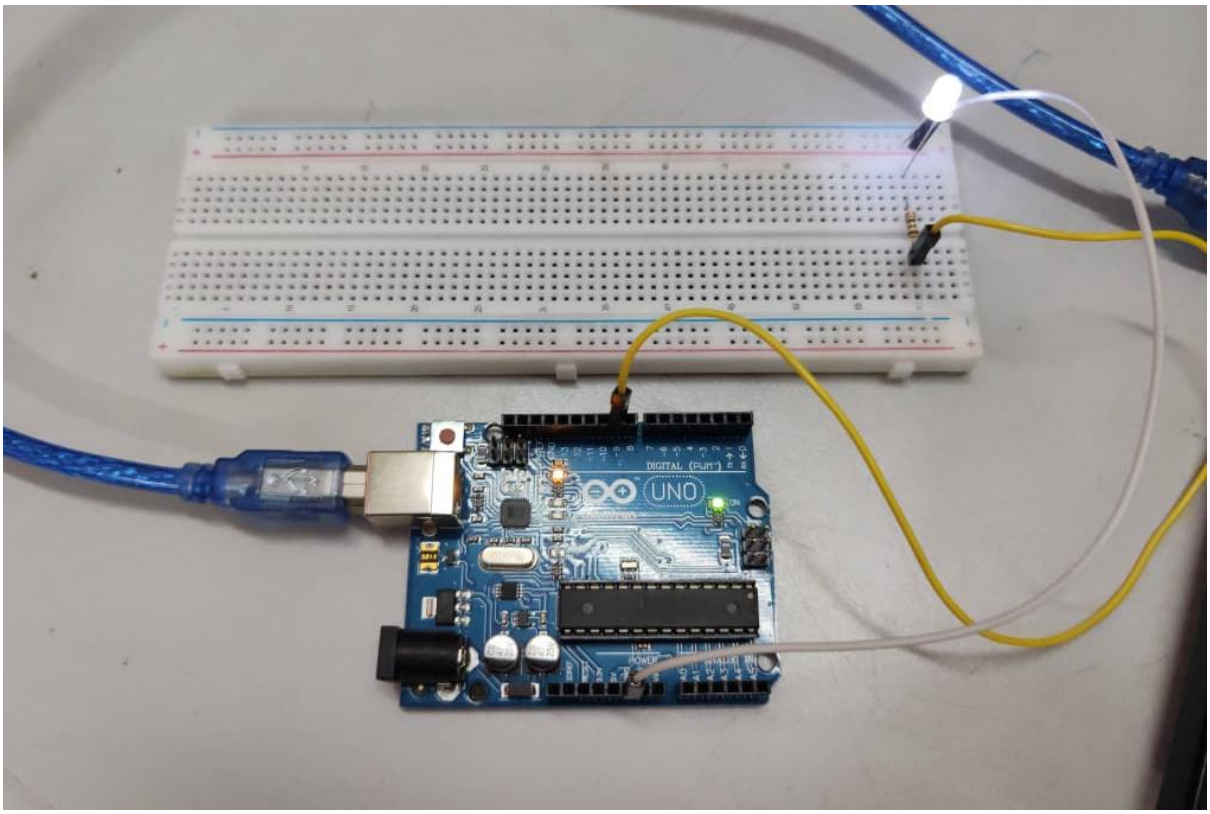
LAB 1 : BLINKING LIGHT

1. Create the circuit.



2. Create the program.

3. Result and Observation.

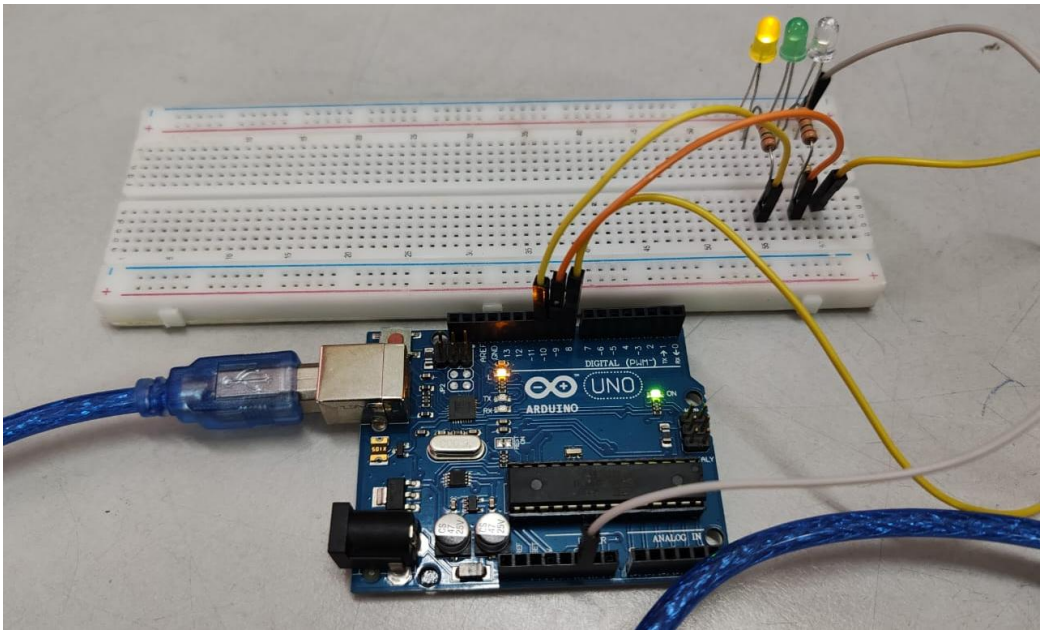


RESULT:
After successfully assembling the circuit and uploading the code into the Arduino board, the LED began to blink in a repeating cycle. It turned on for approximately 10 milliseconds and then turned off for 1000 milliseconds before lighting up again. The blinking pattern was smooth, regular, and consistent throughout the test. This showed that the connection between the Arduino board and the LED was correctly made, and the program uploaded without any syntax or logic errors. The code functioned exactly as intended, demonstrating the ability of the Arduino microcontroller to send digital signals to an external component and control it based on the timing parameters defined in the program. During the experiment, it was observed that even small changes to the delay values in the code affected the blinking speed of the LED, which clearly illustrated the importance of timing functions in programming microcontroller-based systems. The experiment also emphasized how the Arduino IDE provides an easy and reliable platform to write, compile, and upload programs that interact directly with electronic components.

CONCLUSION:
The Blinking Light experiment was a fundamental but highly effective introduction to the Arduino programming environment. It successfully demonstrated how to control a digital output device using simple code commands such as pinMode(), digitalWrite(), and delay(). Through this lab, it became clear that the Arduino microcontroller can precisely execute repetitive instructions, which is essential for automation and embedded systems. The experiment helped strengthen the understanding of how coding and hardware integration work together to achieve a specific output. It also taught that by adjusting parameters in the program, such as delay intervals, one can easily modify the behavior of the LED, allowing customization and flexibility in real-time control systems. Overall, this experiment served as a solid foundation for learning more complex tasks involving sensors, actuators, and real-world applications that depend on accurate timing and logic control.

LAB 2 : RUNNING LIGHT

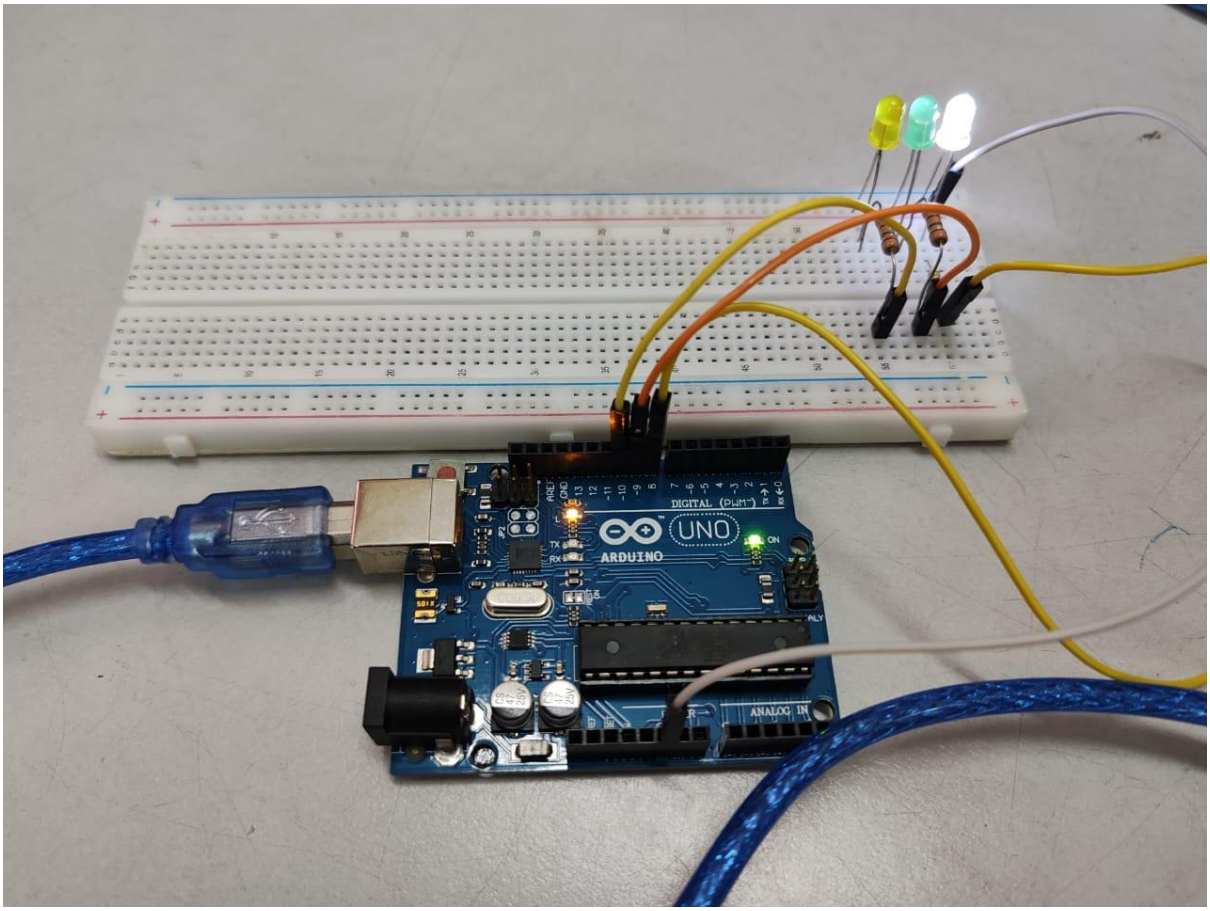
1. Create the circuit.



2. Create the program.

```
// Pin assignments
int redLED = 8;
int greenLED = 9;
int blueLED = 10;

void setup() {
  // Set pins as output
  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(blueLED, OUTPUT);
}

void loop() {
  // Red light ON
  digitalWrite(redLED, HIGH);
  digitalWrite(greenLED, LOW);
  digitalWrite(blueLED, LOW);
  delay(3000); // stay red for 3 seconds

  // Green light ON
  digitalWrite(redLED, LOW);
  digitalWrite(greenLED, HIGH);
  digitalWrite(blueLED, LOW);
  delay(3000); // stay green for 3 seconds

  // Blue light ON (acting as yellow)
  digitalWrite(redLED, LOW);
  digitalWrite(greenLED, LOW);
  digitalWrite(blueLED, HIGH);
  delay(1000); // stay yellow for 1 second
}
```

3. Result and Observation

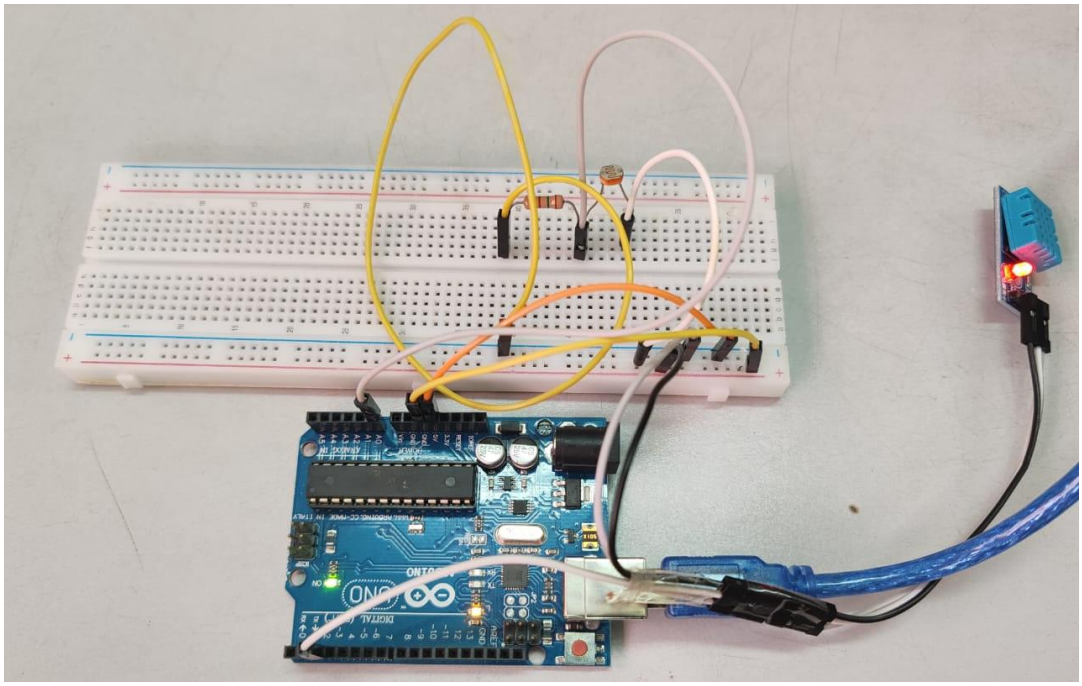


RESULT:
In this experiment, multiple LEDs were arranged in a linear sequence and connected to different output pins on the Arduino board. Once the circuit was properly built and the program was uploaded, the LEDs began to light up one after another in a continuous loop, creating the visual effect of a “running” or “chasing” light. The timing and order of illumination followed the logic programmed into the Arduino sketch, where each LED was turned on for a short period before moving to the next. The pattern repeated continuously without any errors or interruptions, proving that the hardware setup and software logic were working harmoniously. The output on the LEDs clearly showed the effectiveness of using loops and delays in Arduino programming to control multiple outputs in a specific sequence. During observation, it was also found that modifying the delay time or the number of LEDs affected the overall appearance of the running light pattern, allowing for creative customization of the display. The system operated smoothly throughout the experiment, indicating stable voltage distribution and accurate execution of the programmed instructions.

CONCLUSION:
The Running Light lab was a successful demonstration of how to manage and coordinate multiple digital outputs using the Arduino IDE. This experiment provided a deeper understanding of loop structures, particularly the for loop, which enables repetitive actions to be performed efficiently in a single block of code. By experimenting with the timing and order of LED activation, it became clear how the Arduino can be programmed to create visual patterns and sequential effects. The concept learned in this experiment is not limited to lights; it can also be applied to control motors, relays, or any other actuators in a sequence. Furthermore, it introduced the idea of scalability — how a simple running light circuit could be expanded into larger and more complex display systems, such as those used in decorative lighting, progress indicators, or even industrial signaling devices. Overall, this lab reinforced both the logical thinking and the technical wiring skills necessary to design efficient, synchronized output systems using microcontrollers.

LAB 3 : TEMPERATURE, HUMIDITY, AND LIGHT SENSOR

1. Create the circuit.

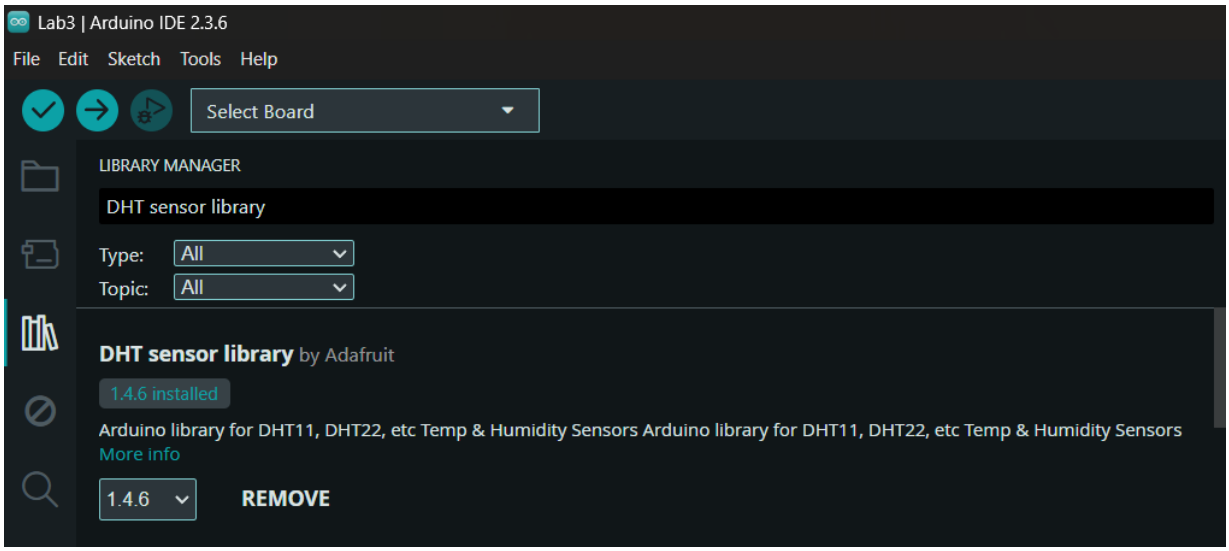


2. Create the program.

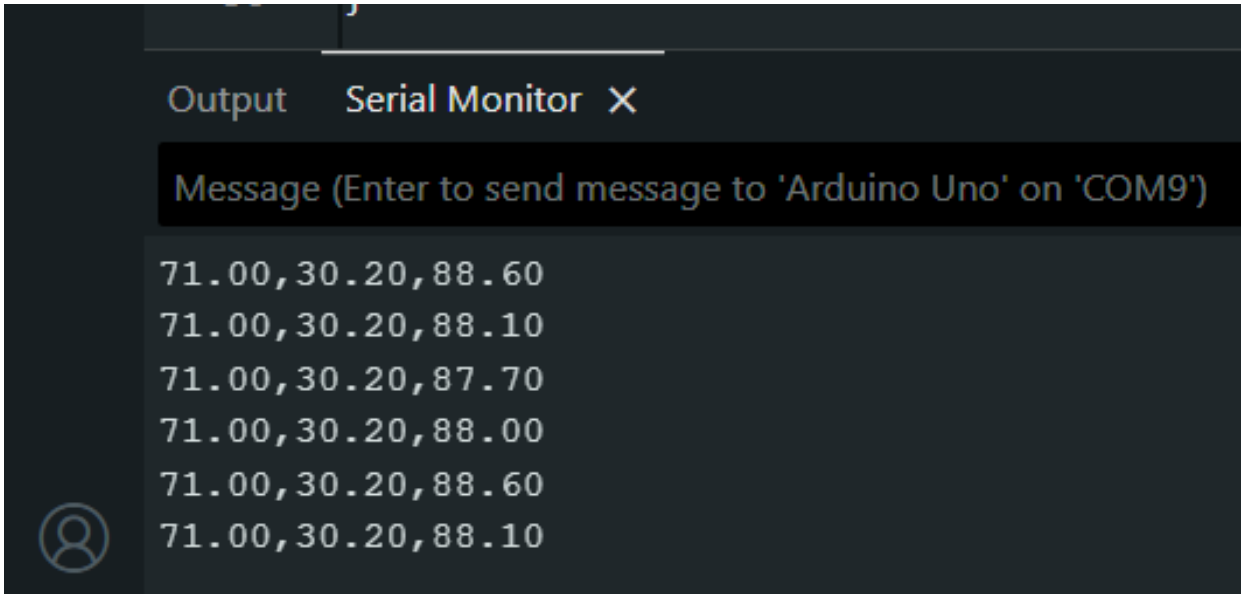
Lab3.ino

```
#include "DHT.h"
#define DHTPIN 2
#define DHTTYPE DHT11
int light_pin = A0;
float light = 0;
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {

  // Wait a few seconds between measurements.
  delay(2000);
  float h = dht.readHumidity();
  // Read temperature as Celsius (the default)
  float t = dht.readTemperature();
  //Getting Light Sensor Reading
  light = analogRead(light_pin);
  light = light/10;


  Serial.print(h);
  Serial.print(",");
  Serial.print(t);
  Serial.print(",");
  Serial.println(light);
}
```

3. Install DHT sensor library on Arduino IDE



4. Result and Observation.



RESULT:

This experiment involved combining multiple sensors to measure environmental parameters such as temperature, humidity, and light intensity. After the circuit was carefully constructed and the DHT sensor library was successfully installed in the Arduino IDE, the program was uploaded to the microcontroller. The Serial Monitor displayed real-time readings from the DHT sensor, showing both temperature (in degrees Celsius) and humidity (in percentage). Simultaneously, the Light Dependent Resistor (LDR) provided continuous analog readings that changed depending on the brightness of the surrounding environment. When the light level increased, the LDR's resistance decreased, causing a noticeable change in the output value, and vice versa when the light level was reduced. The experiment verified that the sensors were functioning correctly and communicating efficiently with the Arduino board. The data displayed on the Serial Monitor updated consistently without delay or error, confirming the stability of both the code and the circuit. These observations demonstrated how the Arduino can collect, process, and display sensor data in real-time, mimicking the functionality of more advanced Internet of Things (IoT) systems.

CONCLUSION:

The Temperature, Humidity, and Light Sensor experiment successfully demonstrated how to integrate multiple sensors with the Arduino to monitor environmental conditions. It provided hands-on experience in sensor interfacing, data acquisition, and serial communication. The experiment showed how important it is to install and use external libraries — in this case, the DHT library — to enable accurate sensor readings and efficient data handling. This lab also highlighted the difference between digital and analog sensors, with the DHT sensor providing discrete data and the LDR generating continuous analog values. By completing this experiment, students gained valuable insights into how sensors can be used in smart systems that automatically respond to environmental changes. This knowledge is directly applicable to real-world applications, such as smart homes, weather monitoring stations, and IoT-based automation systems. Overall, the experiment reinforced key skills in coding, circuit building, and data interpretation, all of which are essential for developing advanced sensor-driven technologies in the future.